ModernTech HR Management System — Presentation Guide

1. Thirty-second introduction

> ModernTech is a full-stack HR management system that brings employee records, attendance, leave, payroll, payslips, timesheets, scheduling, and performance reviews into one application. The frontend is hosted on GitHub Pages, the Express API runs on Railway, and Railway MySQL provides persistent relational storage. JWT authentication and role-based authorization give HR managers and employees different, protected workflows.

---

2. Architecture to explain

```
GitHub Pages frontend
        |
        | HTTPS JSON requests + JWT Bearer token
        v
Railway Express API
        |
        | parameterized SQL through a connection pool
        v
Railway MySQL database
```

- **Frontend:** static HTML, CSS, Bootstrap, JavaScript, Chart.js, and a shared `api.js` gateway.
- **API:** Express routes direct requests to controllers; controllers validate business input; models contain parameterized SQL.
- **Database:** relational tables and foreign keys connect employees to users, attendance, leave, payroll, payslips, timesheets, and reviews.
- **Security:** bcrypt password hashes, eight-hour JWTs, CORS origin restrictions, centralized role authorization, and employee-level record filtering.

3. Recommended five-minute demonstration

Step 1 — Sign in (30 seconds)

- Open `https://nuriyahd.github.io/core-project/`.
- Use `lungile.moyo@moderntech.com` and `ABC12345`.
- Explain that the API verifies a bcrypt hash and returns a signed JWT containing

the user ID, employee ID, and role.

Step 2 — Dashboard (45 seconds)

- Point out that employee, leave, attendance, notification, and payroll values

come from Railway MySQL through four API requests.

- Use the centered search field.
- Open the notification bell and show that it lists real pending leave requests.
- Explain that the payroll visualization becomes a bar for one month and a line

trend when several months exist.

Step 3 — Employees and attendance (45 seconds)

- Open Employees and show the relational department and position data.
- Open Attendance and explain the one-record-per-employee-per-day rule.
- Mention that clock-in and clock-out are protected employee self-service actions.

Step 4 — Time off (45 seconds)

- Show a pending request and the HR approval workflow.
- Explain that employees can submit and view their own requests, while HR can

approve or deny requests.

- Return to the dashboard to show that pending counts and notifications reflect

database state.

Step 5 — Payroll and performance (60 seconds)

- Show payroll totals and individual payslip data.
- Open Performance Reviews and then Review Cycle.
- Move a review from Employee Input to Manager Review and explain the database

mapping: `Draft`, `Submitted`, and `Completed`.

- Refresh the page to prove the selected workflow state persists.

Step 6 — Security close (30 seconds)

- Explain that protected requests carry the JWT in the Authorization header.
- Explain that the API does not trust the page: it independently enforces role

permissions and employee ownership.

- Show `/api/health` as the deployment availability check.

---

4. Key technical decisions

Why use a layered backend?

Routes define endpoints, controllers handle validation and workflow rules, and models own SQL. This separation makes the system easier to test, maintain, and extend without mixing HTTP behavior with database code.

Why JWT authentication?

The frontend and API are hosted on different domains. A JWT supports this stateless architecture: after login, the browser sends a signed token on each protected request and the API verifies it without storing server-side sessions.

Why foreign keys?

Foreign keys prevent orphaned HR records. For example, attendance and reviews must reference real employees, and cascading rules clean up dependent records when an employee is permanently removed.

Why parameterized SQL?

Values are passed separately from SQL statements. This prevents user input from being interpreted as executable SQL and reduces SQL-injection risk.

Why a connection pool?

The pool reuses a limited set of MySQL connections and queues bursts of work. This is more efficient and safer for a hosted database than opening a new connection for every request.

---

5. Questions you are likely to receive

How do you know the dashboard uses real data?

The page calls the deployed `/employees`, `/attendance`, `/leave-requests`, and `/payroll` endpoints. Its counts, lists, notifications, and charts are calculated from those responses rather than hard-coded fixtures.

What prevents an employee from viewing everybody's data?

The API verifies the JWT, restricts non-HR routes centrally, and filters returned records using the employee ID stored in the verified token.

What happens when a token expires?

Tokens expire after eight hours. The API returns an authorization error and the user must sign in again.

How is the frontend allowed to call Railway?

The backend CORS allow-list includes `https://nuriyahd.github.io`. Other browser origins are rejected.

How do review-cycle status changes persist?

The user-friendly UI states map onto the database enum: Employee Input maps to `Draft`, Manager Review maps to `Submitted`, and Final Sign-off maps to `Completed`.

What would you improve next?

- Add automated controller and integration tests to continuous deployment.
- Replace the shared demo password with individual onboarding/reset flows.
- Add database migrations and scheduled backups.
- Add pagination and server-side search for larger employee datasets.
- Add audit events for every sensitive HR change.

---

6. Presentation safety checklist

- Open all required tabs before presenting.
- Confirm Railway and MySQL show **Online**.
- Test login and `/api/health` before the session.
- Use `Ctrl+F5` so GitHub Pages does not show a cached build.
- Never reveal Railway variables, database passwords, or the JWT secret.
- Keep MySQL Workbench closed unless database structure is specifically asked.
- Prepare one pending leave request and one draft performance review for the demo.
- Do not import `database.sql` again before presenting; it drops and recreates tables.

---

7. Strong closing statement

> This project demonstrates more than a collection of pages: it is a deployed, database-backed HR workflow system. The interface reflects live relational data, the API enforces authentication and role boundaries, and core actions such as leave approval, attendance, payroll, and performance progression persist across sessions.